<HLS Programming with FPGAs>

The HLS Book

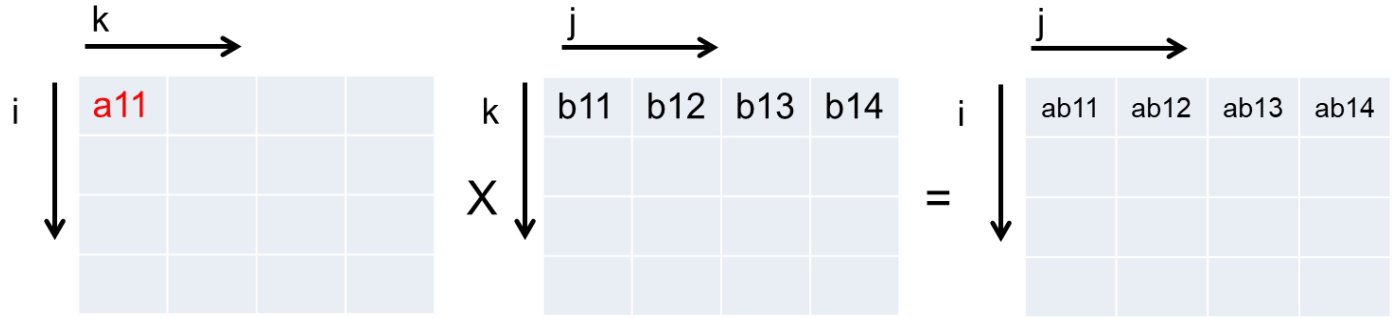
Chapter 7

Dense Matrix Multiplication 3

Choi, Young-kyu

Recap

- Blocked MM helps reduce the DRAM access with data reuse
- Loop ordering and internal memory usage
  - If loop order is k → i → j
    - Internal memory size for A : 1 (∵ reuse distance = 1)
    - Internal memory size for B : M (∵ reuse distance = M)



    - DRAM access order:
      - A : Column traversal ☹, B and C: Row traversal ☺

Recap

- Pipelining & Unrolling

```
.....
  for(int i = 0; i < M; i++) {
#pragma HLS pipeline II=1
    DTYPE Atemp = A[(ib*M+i)*N + kb*M + k];
    for(int j = 0; j < M; j++) {
#pragma HLS unroll
      AB_block[i][j] += Atemp * B_line[j];
  } }
.....
```

- Performance: 47 GOPS (< 128 GOPS ideal)
  - Reason for the low performance : Low effective DRAM BW
  - Needs DRAM optimizations

Learning Objectives

- Increase the effective DRAM bandwidth using a wider bus type
- Explain why using a transposed matrix will bring performance benefits
- Improve the performance with dataflow optimization

Opt2: Wide (Vector) Bus Data Type

- Modify top function argument to 512b (32 shorts)

```
const int DSIZE = 64/sizeof(DTYPE); //number of shorts in 512b(=32)

                                        Can read 512b of data per cycle
void mm(DTYPE* A,  hls::vector<DTYPE, DSIZE> *B,
                   hls::vector<DTYPE, DSIZE> *AB, int N ){
#pragma HLS INTERFACE mode=m_axi bundle=m0 port=A
#pragma HLS INTERFACE mode=m_axi bundle=m1 port=B
#pragma HLS INTERFACE mode=m_axi bundle=m1 port=AB

}
```

- Modify data copy routines
  - Fetching a line of matrix B from DRAM:

```
for(int jj = 0; jj < M/DSIZE; jj++) {
#pragma HLS pipeline II=1
  hls::vector<DTYPE, DSIZE> B_temp
                            = B[((kb*M+k)*N+jb*M)/DSIZE + jj];
  for(int j = 0; j < DSIZE; j++) {
#pragma HLS unroll
    B_line[jj*DSIZE + j] = B_temp[j];
} }
```

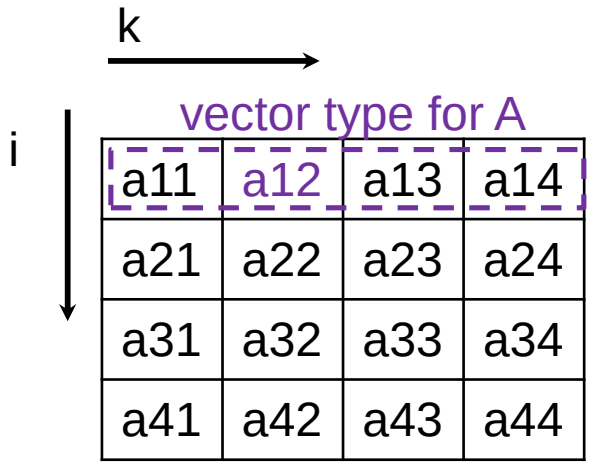
  - Writing entire matrix AB to DRAM:

```
for(int jj = 0; jj < M/DSIZE; jj++) {
#pragma HLS pipeline II=1
  hls::vector<DTYPE, DSIZE> AB_temp;
  for(int j = 0; j < DSIZE; j++) {
#pragma HLS unroll
    AB_temp[j] = AB_block[i][jj*DSIZE + j];
  }
  AB[((ib*M+i)*N+jb*M)/DSIZE + jj] = AB_temp;
} }
```

- Modify data copy routines (…continued)
  - Matrix A:

```
for(int k = 0; k < M; k++) {
  ......
  for(int i = 0; i < M; i++) {
    DTYPE Atemp = A[(ib*M+i)*N + kb*M + k]; // no changes
    for(int j = 0; j < M; j++) {
      AB_block[i][j] += Atemp * B_line[j];
} } }
```

Why?



: since loop order is k → i → j, data copied in a vector (e.g. a12) will be used M iterations later
→ Requires a large internal memory (≈32*M) for matrix A
→ Vector type for A is inefficient for this loop ordering

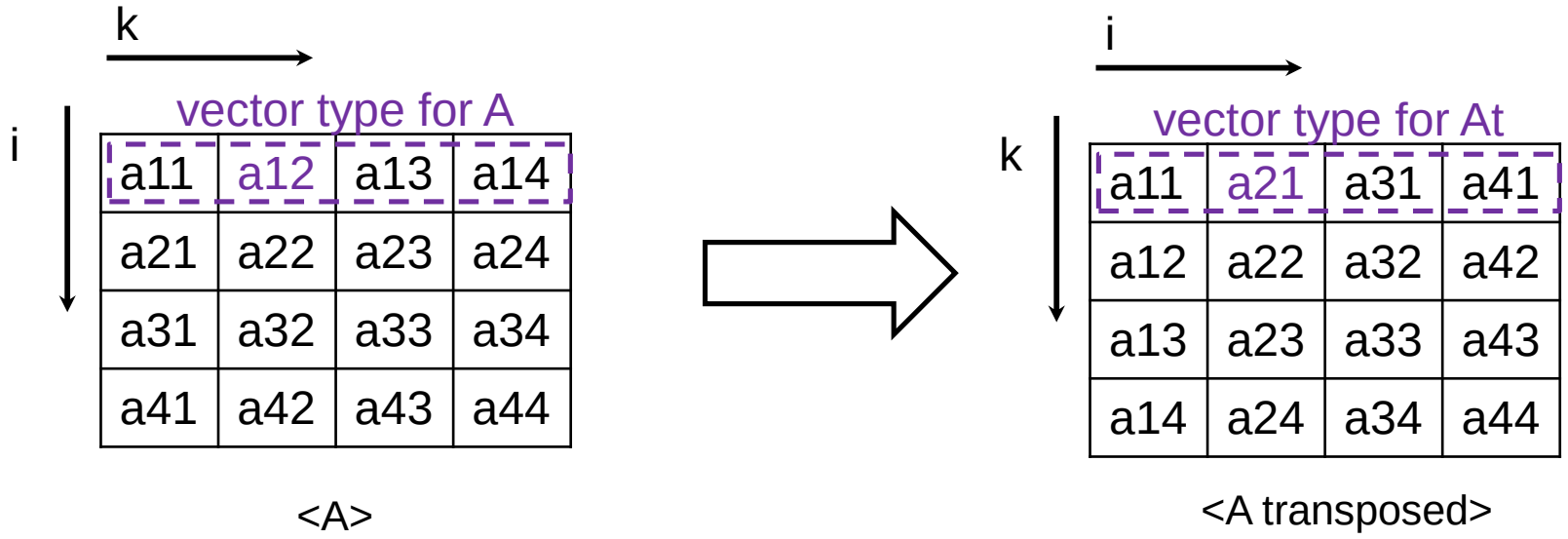
- Performance after using a wide bus data type for B and AB
  - Execution time: 3.2ms
  - Performance: 84 GOPS
  - Comparison:

	Performance (GOPS)	Speedup
Baseline	0.049	1.0X
+ Pipeline/Unroll	47	960X
+ Wide bus type	84	1,700X
Ideal	128	-

  - Decent performance, but there is a room for improvement

Opt3 : Transposed Matrix A

- Optimization idea: Transpose matrix A
  - Benefit 1 : We can fetch data from At in a consecutive address (when loop order is in k → i → j)
  - Benefit 2 : We do not need a large internal memory even if we use a vector type for matrix At



- Modify matrix A data copy & computation routines in the host testbench file
  - mm_sw() in host testbench:
    - Change address indexing of i and k

Non-transposed:

```
for(int i = 0; i < M; i++){
  for(int j = 0; j<M; j++){
    sum = 0;
    for(int k = 0; k < M; k++){
      sum = sum + A[i*M+k] * B[k*M+j];
    }
    AB[i*M+j] = sum;
  }
}
```

Matrix A transposed:

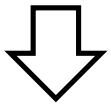
```
for(int i = 0; i < M; i++){
  for(int j = 0; j<M; j++){
    sum = 0;
    for(int k = 0; k < M; k++){
      sum = sum + At[k*M+i] * B[k*M+j];
    }
    AB[i*M+j] = sum;
  }
}
```

- Modify matrix A data copy & computation in the kernel file

Non-transposed:

```
void mm(DTYPE* A, ... ){
  for(int k = 0; k < M; k++) {
    ......
    for(int i = 0; i < M; i++) {
      DTYPE Atemp = A[...i*M+k...];
      for(int j = 0; j < M; j++) {
        AB_block[i][j] += Atemp * B_line[j];
} } } }
```

Code modifications:
1. Transpose address index
2. Change to vector type
3. (optional) Copy a line of At into internal memory



Matrix A transposed:

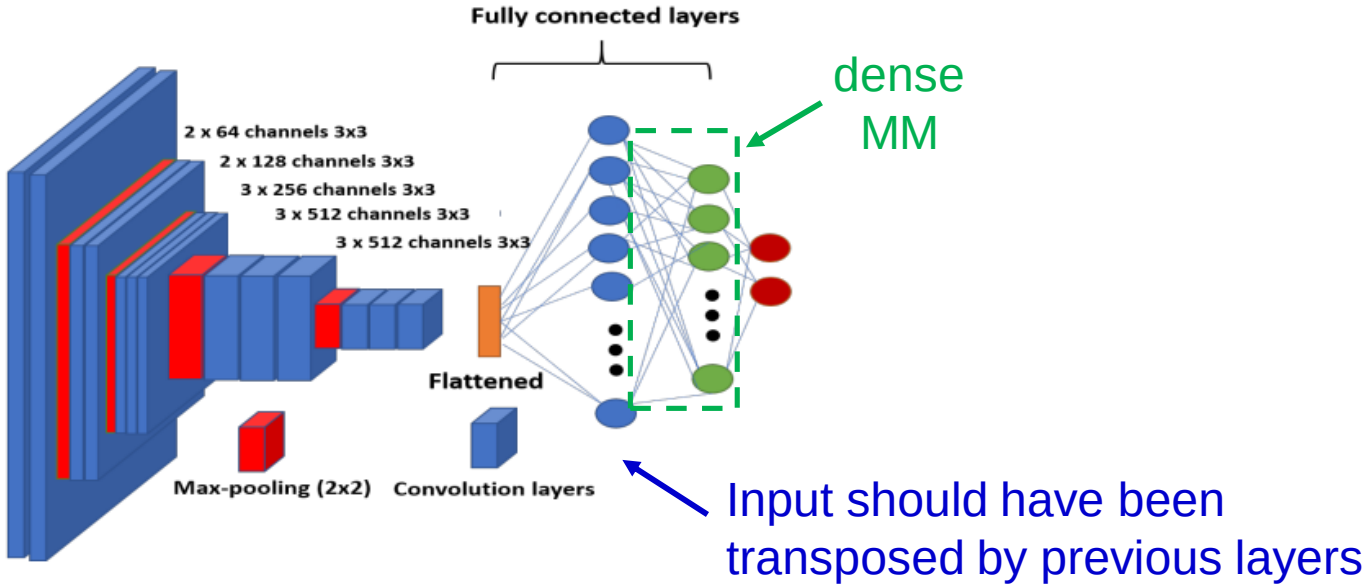
```
void mm(hls::vector<DTYPE, DSIZE> *At, ... ){
  for(int k = 0; k < M; k++) {
    ......                          (A_line[...] = At[...k*M+i...])
    DTYPE A_line[M];
    //copy one line of At into A_line here (using i loop)
    for(int i = 0; i < M; i++) {
      for(int j = 0; j < M; j++) {
        AB_block[i][j] += A_line[i] * B_line[j];
} } } }
```

- Performance after using a transposed matrix A
  - Execution time: 2.7ms
  - Performance: 99 GOPS
  - Comparison:

	Performance (GOPS)	Speedup
Baseline	0.049	1.0X
+ Pipeline/Unroll	47	960X
+ Wide bus type	84	1,700X
+ Transposed A	99	2,020X
Ideal	128	-

- Caution: If input matrix is not already transposed, the DRAM data layout has to be modified before applying this optimization → incurs overhead
- If the previous kernel providing input to dense MM cannot provide a transposed matrix for A, this optimization may NOT be very effective

<VGG16>



Khaleghian, S., et al. "Sea Ice Classification of SAR Imagery Based on Convolution Neural Networks." *Remote Sensing*, 2021.

Opt4 : Dataflow

- Problem : HLS has some difficulty in perfectly overlapping matrix A and B data copy with the MM computation
- Solution : Dataflow optimization
  - Please refer to Part “AWS F1 DRAM Architecture and Optimization”

- Code:
  - Top function:

```
void mm(hls::vector<DTYPE, DSIZE> *At, hls::vector<DTYPE, DSIZE> *B,
                hls::vector<DTYPE, DSIZE> *AB, int M )
{
#pragma HLS INTERFACE mode=m_axi bundle=m0  port=At
#pragma HLS INTERFACE mode=m_axi bundle=m1  port=B
#pragma HLS INTERFACE mode=m_axi bundle=m1  port=AB

#pragma HLS DATAFLOW
```

FIFO interface between functions

```
  hls::stream< hls::vector<DTYPE, DSIZE> > AStreamWide("AStreamWide");
  hls::stream<DTYPE> AStream("AStream");
  hls::stream< hls::vector<DTYPE, DSIZE> > BStream("BStream");
  hls::stream< hls::vector<DTYPE, DSIZE> > ABStream("ABStream");

  ReadAt(At, AStreamWide, M);                  // reads matrix At
  ChangeA_Rate(AStreamWide, AStream, M);       // optional, see next slide
  ReadB(B, BStream, M);                        // reads matrix B
  Comp(AStream, BStream, ABStream, M);         // compute MM & write AB
  WriteAB(ABStream, AB, M);                    // writes matrix AB
```

Functions executed in parallel

```
}
```

- Code:
  - ChangeA_Rate() function (optional) :
    - Purpose : Changes matrix A vector type to single elements
    - Benefit : Reduces burden on Comp() to extract A elements
    - Implementation:

```
void ChangeA_Rate(hls::stream<hls::vector<DTYPE,DSIZE> > & AStreamWide,
                  hls::stream<DTYPE> & AStream,  int N )
{
  for(int ib = 0; ib < N/M; ib++) {
    for(int jb = 0; jb < N/M; jb++) {
      for(int kb = 0; kb < N/M; kb++) {
        for(int k = 0; k < M; k++) {
          for(int ii = 0; ii < M/DSIZE; ii++) {
            hls::vector<DTYPE, DSIZE> A_temp = AStreamWide.read();
            for(int i = 0; i < DSIZE; i++) {
#pragma HLS pipeline II = 1
              AStream.write(A_temp[i]);
} } } } } } }
```

Exercise

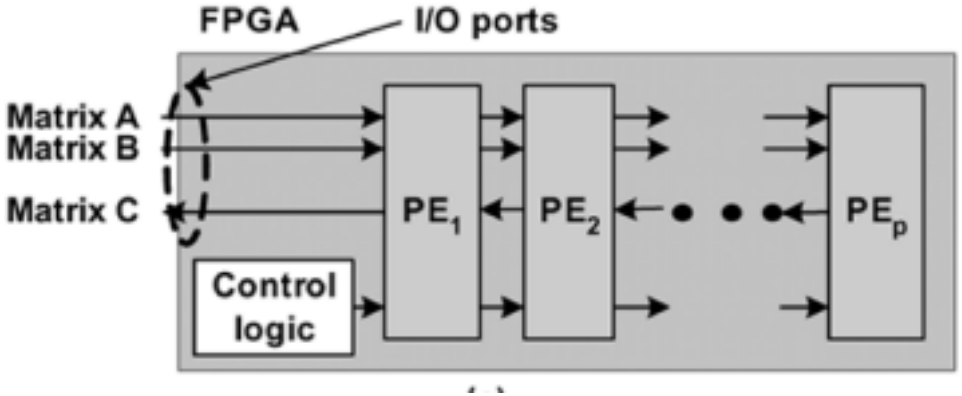
- Please implement the rest of the functions for dataflow-optimized dense matrix multiplication

- Performance after using dataflow optimization
  - Execution time: 2.38ms
  - Performance: 113 GOPS
  - Comparison:

	Performance (GOPS)	Speedup
Baseline	0.049	1.0X
+ Pipeline/Unroll	47	960X
+ Wide bus type	84	1,700X
+ Transposed A	99	2,020X
+ Dataflow	113	2,310X
Ideal	128	-

Systolic Arrays

- Data distribution from AXI controllers to PEs (MAC units) becomes complicated when the # of PE is large → a large drop in the clock frequency
- Systolic arrays:
  - Passes data to neighbor PEs in rhythmically
  - Clock frequency improvement with local connections



  - Ex) Google TPU

J. Jang, S. B. Choi, and V. K. Prasanna. "Energy-and time-efficient matrix multiplication on FPGAs." *IEEE TVLSI 2005*

Further Readings

- J. Jang, S. B. Choi, and V. K. Prasanna. "Energy-and time-efficient matrix multiplication on FPGAs." *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 13.11 (2005): 1305-1319.
- J. Wang, L. Guo, and J. Cong. "AutoSA: A Polyhedral Compiler for High-Performance Systolic Arrays on FPGA." *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 2021.

Summary

- We should utilize wide (vector) bus data type to increase the effective DRAM bandwidth
- Depending on the loop order, using a transposed matrix for A could improve the effective DRAM BW
- Computation and memory access is better overlapped after applying dataflow optimization
- Systolic array architecture improves the frequency when using a large amount of PEs